

BUILDING DATABASE APPLICATIONS

Staff Graded Assignment - 2

Library Management System

University Edition

SUBMITTED BY

Dev Kshirsagar

STUDENT ID

2024EB01165

GITHUB REPOSITORY

github.com/DevK17/library-management-app

1. Project Overview

The Library Management System (University Edition) is a full-stack web application developed using Java 25 and the Spring Boot 3.2.5 framework. The system provides complete Create, Read, Update, and Delete (CRUD) functionality for two primary domain entities -- Authors and Books -- connected through a formal One-to-Many relational model. The application targets an academic library context, pre-loading a curated dataset of university-relevant textbooks and academic authors on startup.

Technology Stack

Component	Technology / Version
Language	Java 25
Framework	Spring Boot 3.2.5
Persistence	Spring Data JPA (Hibernate 6)
Database	H2 In-Memory Database
Build Tool	Apache Maven
View Layer	JavaServer Pages (JSP)
Testing	JUnit 5, MockMvc (Spring Boot Test)
Version Control	Git / GitHub

Design Decisions

The H2 in-memory database was selected to eliminate external database configuration overhead during development and assessment. Hibernate's *create-drop* DDL strategy ensures the schema is generated fresh on each startup and torn down on shutdown, guaranteeing a predictable state at all times. Spring Data JPA was chosen over raw JDBC to leverage type-safe repository abstractions, reducing boilerplate while maintaining full control over query behaviour.

The view layer is implemented with JSP files rendered through Spring MVC's `InternalResourceViewResolver`. The UI adopts an Academic Navy colour scheme -- a deep navy header with standard table borders and breadcrumb navigation -- presenting a clean, professional appearance appropriate for an academic institution. Constructor injection is used throughout the service and controller layers in preference to field injection, aligning with the Spring Framework's recommended practices for testability.

2. Entity Relationship Design

The data model is composed of two JPA-mapped entities: **Author** and **Book**. These entities participate in a strict One-to-Many (1:N) relationship -- one Author may be associated with zero or

more Books, but each Book must reference exactly one Author. The relationship is enforced both at the application layer (through JPA annotations) and at the database layer (through a foreign key constraint on the `books.author_id` column).

Author Entity -- Table: authors

Column	Java Type	JPA / DB Constraint	Description
id	Long	PK, AUTO_INCREMENT, NOT NULL	Surrogate primary key
name	String	NOT NULL	Full name of the author
email	String	NOT NULL, UNIQUE	Institutional email address
nationality	String	NOT NULL	Author's country of origin
books	List<Book>	@OneToMany (mapped by author_id)	Lazy-loaded collection of books

Book Entity -- Table: books

Column	Java Type	JPA / DB Constraint	Description
id	Long	PK, AUTO_INCREMENT, NOT NULL	Surrogate primary key
title	String	NOT NULL	Full title of the book
genre	String	NOT NULL	Genre or subject area
publishedYear	int	NOT NULL	Four-digit publication year
isbn	String	NOT NULL	ISBN-13 identifier
author_id	Long (FK)	NOT NULL, FK -> authors(id)	Foreign key to Author

Relationship Constraints

The **Author** side of the association is annotated with **@OneToMany(mappedBy = "author", cascade = CascadeType.ALL, orphanRemoval = true)**. The `cascade = ALL` directive propagates all persistence operations from parent to children, while `orphanRemoval = true` ensures that any Book entity removed from an Author's collection is automatically deleted from the database, maintaining referential integrity without explicit delete calls.

The **Book** side uses **@ManyToOne(fetch = FetchType.LAZY)** with **@JoinColumn(name = "author_id", nullable = false)**. Lazy fetching is specified on the Many side to avoid unnecessary joins when listing books without requiring author detail. The `nullable = false` constraint on the join column mirrors the NOT NULL constraint enforced at the DDL level, preventing orphan book records.

The `email` column on the Author table carries a **UNIQUE** constraint, enforced via **@Column(nullable = false, unique = true)**. This models the real-world expectation that each institutional author holds a unique email address within the system.

3. Implementation Details

3.1 Entity Classes

Both **Author.java** and **Book.java** are annotated with **@Entity** and **@Table** to register them with the Hibernate JPA provider. Each entity carries a surrogate primary key of type *Long*, generated using the *IDENTITY* strategy -- appropriate for H2's auto-increment column support. All domain fields are declared with explicit **@Column** annotations to enforce NOT NULL constraints at the persistence layer rather than relying solely on application-level validation. A no-argument constructor is provided on each entity as required by the JPA specification.

3.2 Repository Layer

AuthorRepository and **BookRepository** both extend **JpaRepository<T, Long>**, inheriting the full set of CRUD operations and paging/sorting support from Spring Data. *BookRepository* additionally declares a derived query method **findByAuthorId(Long authorId)**, which Spring Data resolves to a JPQL query at startup without any implementation code. This provides filtered retrieval of books belonging to a specific author, used by the Author detail view.

3.3 Service Layer

AuthService and **BookService** act as the transactional boundary between the controller and repository layers. Each service is annotated with **@Service** and uses constructor injection to receive its repository dependency. Service methods wrap repository calls in coherent units of work and apply business rules -- for example, verifying that an entity exists before attempting an update, throwing *IllegalArgumentException* if not. This separation ensures that controllers remain thin and that business logic is not duplicated across multiple entry points.

3.4 Controller Layer and Exception Handling

AuthorController and **BookController** are annotated with **@Controller** and **@RequestMapping** to map HTTP verbs to handler methods. Both controllers accept a **RedirectAttributes** parameter in POST and DELETE handlers, enabling flash attributes to survive the post-redirect-get cycle and display feedback messages to the user on the subsequent page load.

Foreign key constraint violations are handled by catching **DataIntegrityViolationException** directly in the controller. When a user attempts to delete an Author that still has Books assigned, the JPA layer propagates this exception. The catch block in *AuthorController.deleteAuthor()* intercepts it before it reaches the default error handler, adds a human-readable message via *redirectAttributes.addFlashAttribute("errorMessage", ...)*, and redirects back to the authors list. This prevents an unhandled HTTP 500 response and preserves the user's workflow.

3.5 Data_INITIALIZER

DataInitializer.java, implemented as a Spring **@Component** extending **CommandLineRunner**, executes once immediately after the application context is fully initialised. It seeds the database with five academic authors (Dr. Thomas Reynolds, Prof. Sarah Mitchell, James Weber, Dr. Anita Patel, and a test entry) and ten books spanning Textbook, Technology, Software Engineering, and Reference genres. The seed data uses realistic ISBN-13 values and institutional email addresses.

Because the JPA DDL strategy is set to *create-drop*, this data is recreated fresh on every application startup.

4. Application Screenshots

The following screenshots were captured from the running application and illustrate the primary user interface screens. All views share the Academic Navy header layout with breadcrumb navigation.

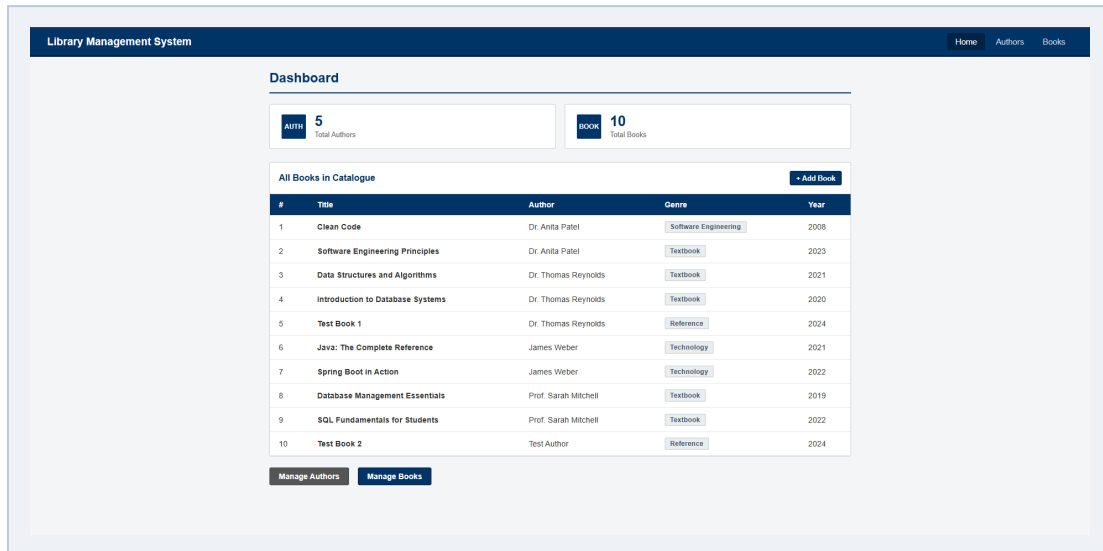


Figure: Dashboard showing the book catalogue and seeded textbook data.

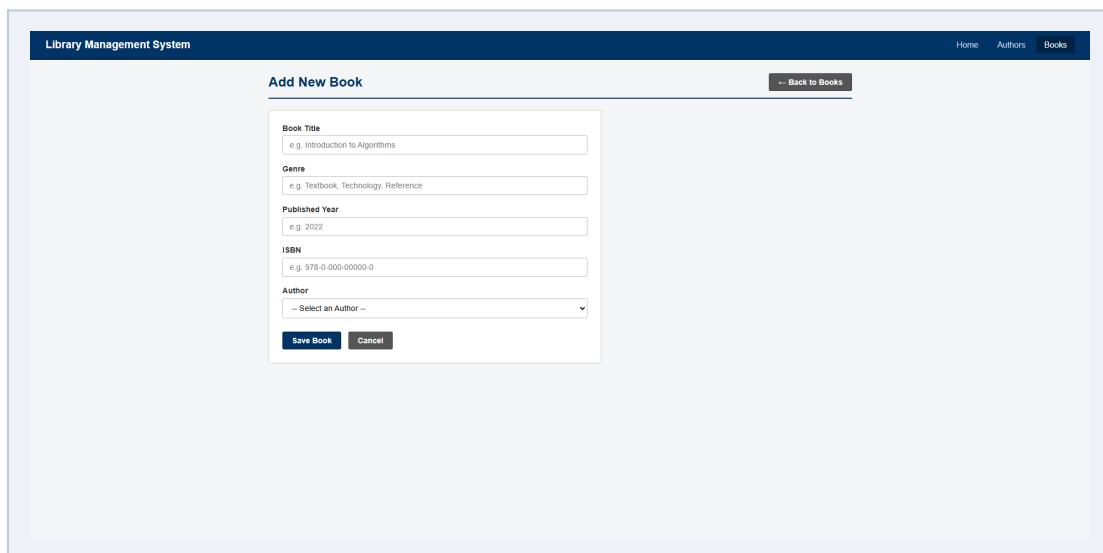


Figure: Book creation interface with author selection dropdown.

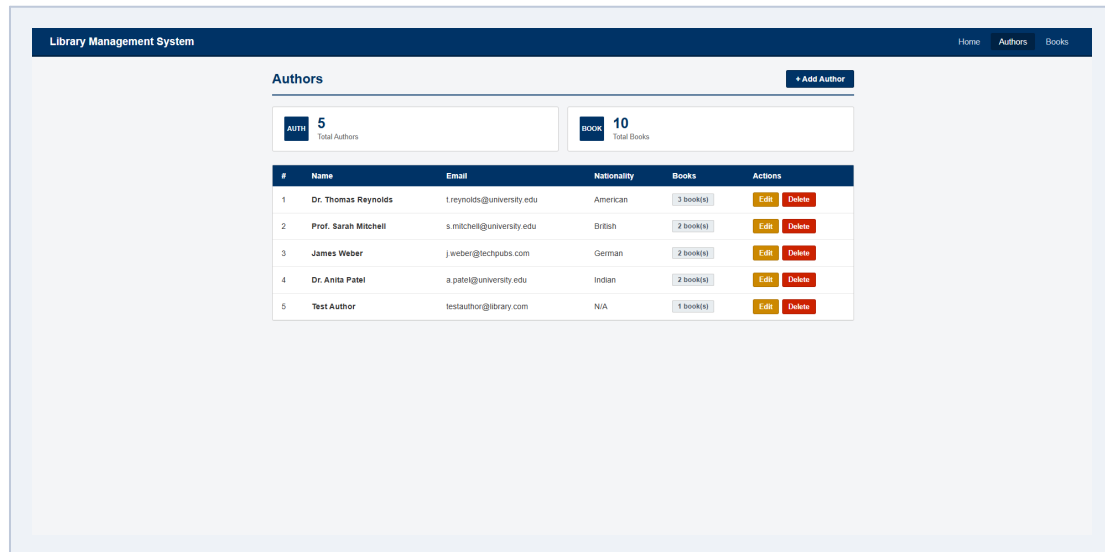


Figure: Author directory displaying existing records and available actions.

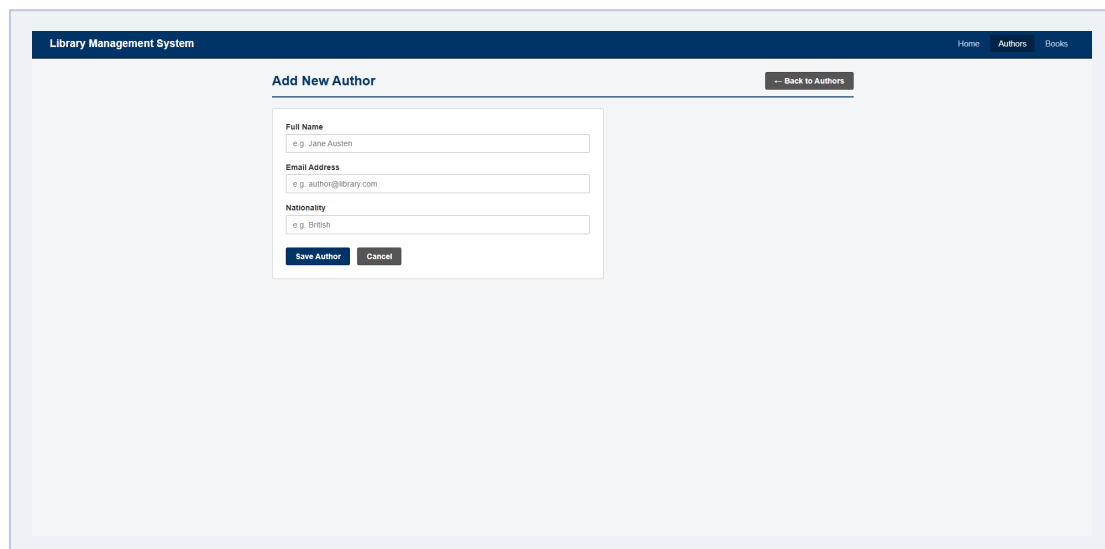


Figure: Input form for new Author record creation.

5. Testing

The test suite comprises **33 tests** distributed across four test classes, all of which pass without error or warning. Tests are written using **JUnit 5** and **MockMvc** for controller verification, with **@DataJpaTest** slices for the repository layer.

Test Class	Scope	Count
AuthorRepositoryTest	Repository / H2 integration	9
BookRepositoryTest	Repository / H2 integration	8
AuthServiceTest	Service layer (Mockito mocks)	8

BookServiceTest	Service layer (Mockito mocks)	8
Total		33

Repository tests use the **@DataJpaTest** annotation, which configures an in-memory H2 database, scans only JPA-relevant components, and wraps each test in a transaction that is rolled back after completion. Service tests apply **@ExtendWith(MockitoExtension.class)** and inject mock repositories, verifying that service methods call the correct repository operations and propagate exceptions appropriately.

```

Hibernate: insert into authors (author_id,name,nationality,id) values (?,?,?,?)
Hibernate: insert into books (author_id,genre,isbn,published_year,title,id) values (?,?,?,?,?,default)
Hibernate: select b1_0.id,b1_0.author_id,b1_0.genre,b1_0.isbn,b1_0.published_year,b1_0.title from books b1_0 where upper(b1_0.genre)=upper(?)
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.090 s -- in com.library.repository.BookRepositoryTest
[INFO] Running com.library.service.AuthorServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.199 s -- in com.library.service.AuthorServiceTest
[INFO] Running com.library.service.BookServiceTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.063 s -- in com.library.service.BookServiceTest
[INFO] Results:
[INFO] Tests run: 33, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 7.576 s
[INFO] Finished at: 2026-05-03T08:11:21+05:30
[INFO]
PS C:\Users\kshir\Desktop\library-management-app>

```

Figure: Terminal execution output confirming all 33 tests passing successfully.

6. Application Configuration

All runtime configuration is declared in **src/main/resources/application.properties**. The key properties are reproduced below:

```

# Application identity
spring.application.name=library-management-app

# H2 in-memory datasource
spring.datasource.url=jdbc:h2:mem:librarydb;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=FALSE
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# JPA / Hibernate
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true

# H2 web console
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# JSP view resolver
spring.mvc.view.prefix=/WEB-INF/views/
spring.mvc.view.suffix=.jsp

# Embedded server

```

```
server.port=8080
```

Running the Application

The application is built and launched with a single Maven command from the project root directory. Because *create-drop* DDL mode is active, the schema and seed data are re-created on every startup.

```
mvn spring-boot:run
```

Once started, the application is accessible at **http://localhost:8080**. The H2 database console is available at **http://localhost:8080/h2-console** -- connect using JDBC URL *jdbc:h2:mem:librarydb*, username *sa*, and an empty password to inspect the *authors* and *books* tables directly during a running session.

7. Challenges and Solutions

Challenge 1: Enabling JSP Support in Spring Boot 3

Problem: Spring Boot 3.x, when using the embedded Tomcat server, does not include JSP rendering support by default. The standard starter dependencies do not pull in the Jasper JSP compiler, meaning that JSP view files resolve to 404 responses or are served as raw text rather than rendered HTML.

Solution: The **tomcat-embed-jasper** dependency must be added explicitly to *pom.xml* with *provided* scope. Both *spring.mvc.view.prefix* and *spring.mvc.view.suffix* must be set in *application.properties* to configure the *InternalResourceViewResolver*. JSP files must reside under *src/main/webapp/WEB-INF/views/* -- placing them under *src/main/resources* does not work with the embedded Tomcat servlet context. Identifying this combination of requirements involved systematic verification against Spring Boot 3.x documentation and the *tomcat-embed-jasper* release notes.

Challenge 2: Preventing HTTP 500 on Foreign Key Constraint Violations

Problem: When a user attempts to delete an Author record that still has one or more Books referencing it, JPA propagates the underlying SQL constraint violation as a *DataIntegrityViolationException*. Without explicit handling, Spring MVC escalates this to an unhandled HTTP 500 Internal Server Error, which disrupts the user's session and provides no actionable feedback.

Solution: A try-catch block was added in **AuthorController.deleteAuthor()** to catch *DataIntegrityViolationException* explicitly. On catching the exception, a human-readable error message is written to a flash attribute via *RedirectAttributes.addFlashAttribute("errorMessage", ...)* and the response is redirected to the authors list page. The JSP template reads the flash attribute on the next render and displays it in an error banner. This pattern keeps HTTP responses in the 3xx range and communicates the constraint issue to the user without exposing internal stack traces.

Challenge 3: Configuring .gitignore to Exclude Build Artifacts

Problem: Committing Maven build output (the *target/* directory), IDE-specific metadata files, and compiled *.class* files to the repository inflates repository size, introduces machine-specific paths into version control, and causes merge conflicts on files that carry no meaningful change history.

Solution: A comprehensive **.gitignore** file was configured at the project root covering: Maven build output (*target/*), IntelliJ IDEA project metadata (*.idea/*, **.iml*), Eclipse project files (*.project*, *.classpath*, *.settings/*), OS-level metadata (*.DS_Store*, *Thumbs.db*), and compiled class files (**.class*). The *application.properties* file, which contains no secrets in this project, is intentionally tracked so that reviewers can run the application without additional configuration.

8. Project Structure

The project follows the standard Maven directory layout. Source code is organised under the **com.library** base package, partitioned by layer.

```
library-management-app/
|-- pom.xml
|-- .gitignore
+-- src/
|   |-- main/
|   |   |-- java/
|   |   |   +-- com/library/
|   |   |       |-- LibraryApplication.java      (Spring Boot entry point)
|   |   |       |-- entity/
|   |   |           |-- Author.java              (@Entity -- authors table)
|   |   |           +-- Book.java                (@Entity -- books table)
|   |   |       |-- repository/
|   |   |           |-- AuthorRepository.java     (JpaRepository)
|   |   |           +-- BookRepository.java       (JpaRepository)
|   |   |       |-- service/
|   |   |           |-- AuthorService.java        (Business logic -- Authors)
|   |   |           +-- BookService.java          (Business logic -- Books)
|   |   |       |-- controller/
|   |   |           |-- HomeController.java        (GET / redirect)
|   |   |           |-- AuthorController.java      (CRUD -- /authors/**)
|   |   |           +-- BookController.java        (CRUD -- /books/**)
|   |   |       +-- runner/
|   |   |           +-- DataInitializer.java       (CommandLineRunner -- seed data)
|   |   |-- resources/
|   |   |   +-- application.properties            (H2, JPA, MVC, server config)
|   |   +-- webapp/
|   |       +-- WEB-INF/
|   |           +-- views/
|   |               |-- layout/
|   |               |   +-- header.jsp            (Shared navigation header)
|   |               |   |-- home.jsp              (Landing page)
|   |               |   |-- books/
|   |               |       |-- list.jsp           (Book catalogue view)
|   |               |       |-- add.jsp            (Add book form)
|   |               |       +-- edit.jsp          (Edit book form)
|   |               +-- authors/
```

```
|           |-- list.jsp           (Author directory view)
|           |-- add.jsp            (Add author form)
|           |-- edit.jsp           (Edit author form)
|           +-- books.jsp          (Author-specific book list)
+-- test/
    +-- java/
        +-- com/library/
            |-- repository/
                |-- AuthorRepositoryTest.java (9 @DataJpaTest tests)
                |-- BookRepositoryTest.java   (8 @DataJpaTest tests)
            +-- service/
                |-- AuthorServiceTest.java    (8 Mockito unit tests)
                +-- BookServiceTest.java      (8 Mockito unit tests)
```